AI 自迭代的小游戏 Agent：2周上线6款小游戏，3天迭代12款小游戏

原创 营销&交易技术 大淘宝技术 2026年9月22日 23:34 浙江 6人

本文介绍了一套覆盖小游戏从选型、生产到上线迭代全链路的自迭代 Agent 系统。针对单点 AI 提效无法带动整体业务提速的痛点，该系统将分散的 AI 能力串联，由人定义北极星指标与行动边界并审核关键方案，AI 负责策略、开发与数据归因。架构上分为数据感知、运营 Agent、生产 Agent 及游戏基础工程等层级，通过 LangGraph 与 Claude Agent SDK 协同，实现多 Agent 上下文流转与决策闭环。实践表明，该系统两周内可上线 6 款小游戏，最快 2 天完成全流程；运营侧 12 款游戏迭代的正向率超 95%。未来计划将其扩展至更广义的互动玩法，打造可持续探索与优化的互动服务。

01 引言

过去一年，AI Coding、生图、数据分析和 Agent 快速进入软件研发的各个环节。今年 1 月，我们跑通了 AI Coding、生图和关卡生成的一套可复用的小游戏 AI 协同流程和技术基础能力建设。但这里还有一个问题没有解决：**每个环节都在变快，完整的业务流程却没有同比提速**。

这促使我们继续追问：如果 AI 已经能够理解**策划、生成美术、修改代码、读取数据并执行实验**，那么整个游戏生命周期中，还有多少工作需要人持续推动？AI 能否不只帮助我们完成某个环节，而是参与一款游戏从生产、上线到持续迭代的全过程？我们围绕这个问题**搭建了一套覆盖小游戏选型、策划、美术、音频、开发、关卡生产、发布、AB 实验、效果归因和再次迭代的系统**。人在其中主要保留两项职责：定义北极星指标与行动边界，审核关键方案。

这篇文章重点介绍我们如何把分散在各个环节的 AI 能力连接起来，让游戏从创意走到线上，并在真实数据的反馈下自迭代。先放一下我们这套自迭代 Agent 上线验证的初步效果。

02 为什么要做一套自迭代的 Agent

生产侧：单点提效无法支持游戏快速上线

即使 AI Coding、生图和关卡生成的单点效率已经明显提升，上线一款新小游戏仍需 2～3 周。因为每个环节都需要人工反复确认：

- **从创意到原型**：最快也需要 6 天，而小游戏研发周期重度依赖原型。
- **能力孤岛**：策划、美术、代码、关卡各环节的 AI 工具零散割裂。

运营侧：纯人工无法高效同时运营多个游戏

Agent 已经能够按天上线小游戏，但这只是起点，还要可持续运营。而运营环节暴露出与当初生产环节相似的问题：

- **经验难以复用**：不同的运营同学可能对相同的数据给出完全不同的归因，经验往往停留在个人手里。
- **分析不够精细**：10+ 款游戏，叠加场域、游戏、关卡三级数据，并且每日更新，数据量庞大。
- **结论缺少验证**：无论 AI 还是人都能写出论证完整的分析报告，但方案上线后未必对核心指标有正向收益。
- **归因缺乏统一上下文**：数据、代码、关卡配置和变更历史分散在不同系统中，分析时很难完整追溯当时发生了什么。

因此这不是一个简单的单链路 Workflow 问题。生产 Agent 依赖多环节上下文传递，运营 Agent 则依赖持续归因。

如何解决：上下文流转与决策闭环

生产闭环——解决上下文流转：

- 玩法调研、策划方案、资产清单、关卡数据结构进入多 Agent 的全局状态，下游 Agent 直接读取，不再重复获取或依赖人工搬运。
- 用户中途修改需求时，由 Producer 统一判断影响范围，并逐级路由到需要重做的下游环节。

迭代闭环——处理数据驱动决策：

- **感知状态**：读取三级报表、AB 实验快照、变更记录和版本时间线，感知当前状态；
- **理解目标**：对齐北极星指标与行动边界；
- **拆解决策**：判断每日迭代优先级；
- **调用能力**：按任务召唤专家角色，强制加载任务手册和既有知识；
- **产出方案**：产出迭代提案或新游戏立项书；
- **读取反馈**：读取 AB 实验回流数据，到期后对照预期收益结算；
- **沉淀经验**：将归因结论写入知识库与记忆，沉淀迭代经验。

整套系统的分工可以概括为：人定义北极星指标和行动边界，并负责审核；AI 负责策略、开发和数据归因。

03 小游戏自迭代 Agent 全链路架构

L1 数据感知层把前一天的数据、变更记录和游戏代码整理成可信上下文；L2 运营 Agent 判断当天最值得改哪款游戏，产出带预期的待评审方案；人工审核后，方案交给 L3 生产 Agent，并在 L4 游戏容器与模板中变成可上线代码；L5 运营 Agent 负责设置门禁、沉淀知识，并在 AB 实验到期时回收结果。完成一次迭代后，系统重新回到 L1，继续下一轮迭代。

我们没有做一个包揽所有能力的“超级 Agent”，而是把判断和执行拆开：

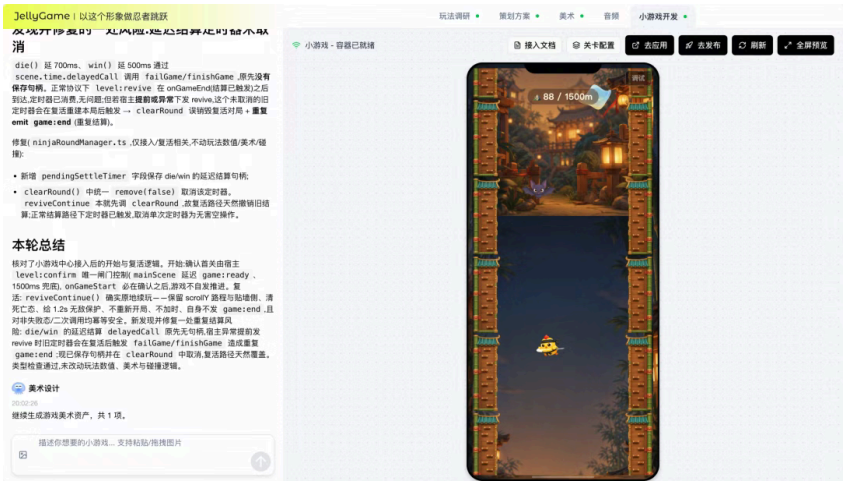
- 生产 Agent 接收已经确认的方案，完成策划、美术、音效、代码和关卡生产。
- 运营 Agent 读取业务报表、AB 实验、变更记录和历史记忆，负责问题归因、优先级判断和决定应该改什么。

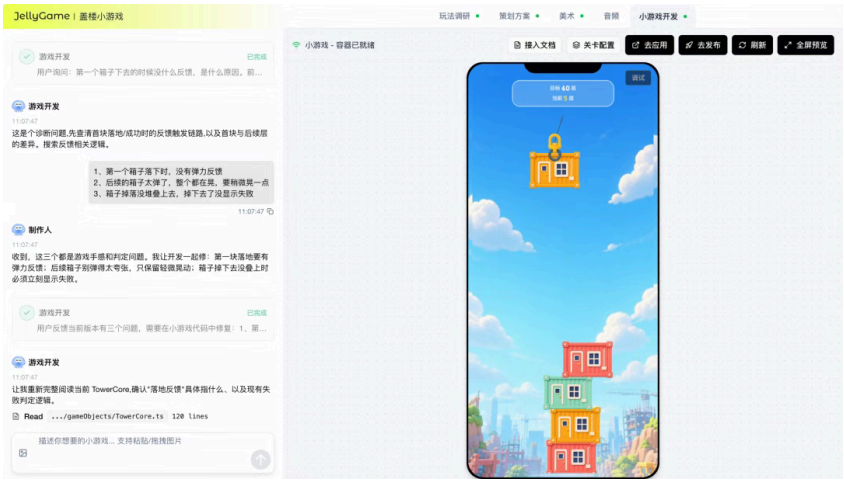
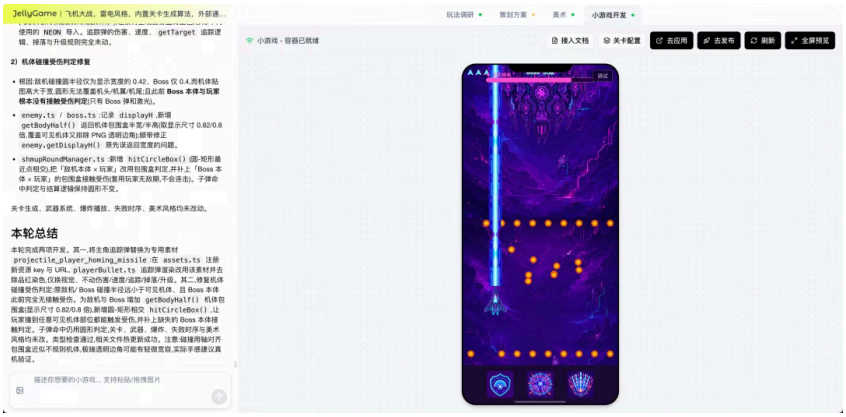
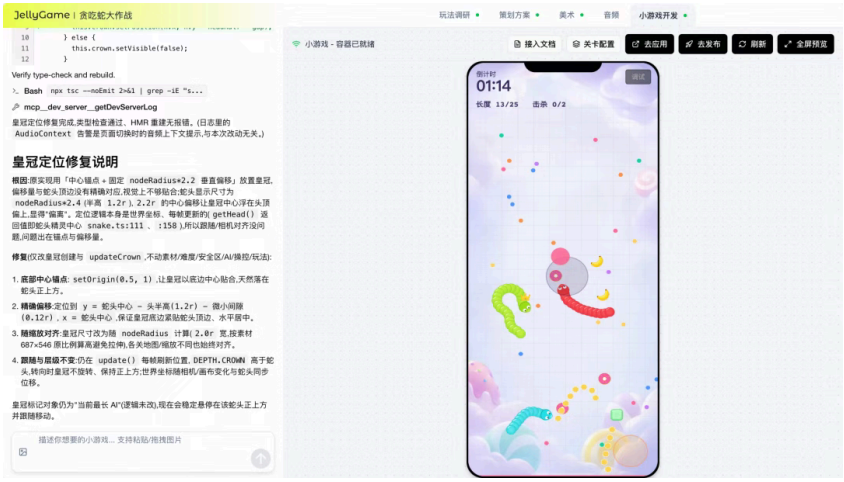
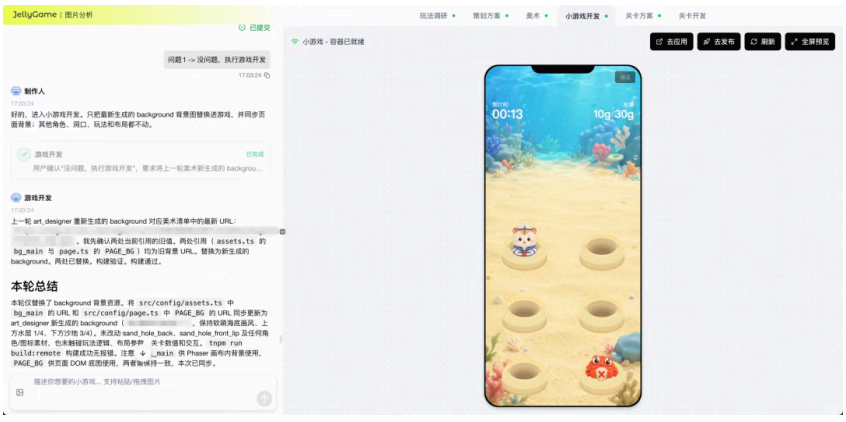
运营 Agent 持续积累业务指标、实验结果和决策依据，生产 Agent 则沉淀游戏架构、资产规范和开发经验，职责清晰，虽隔离但可以互相查阅上下文。

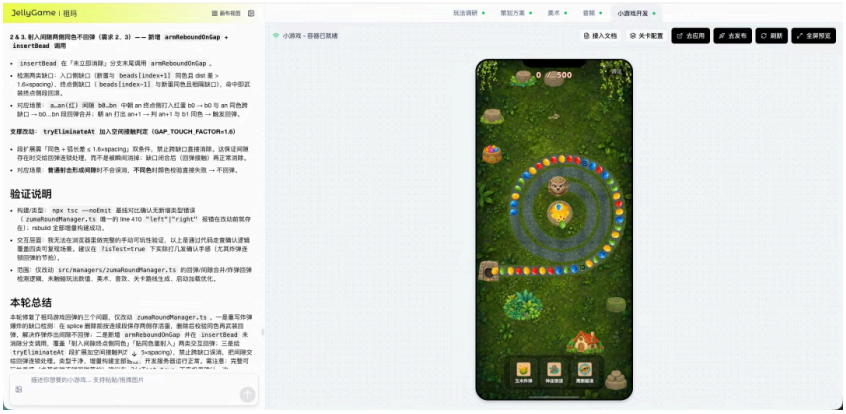
04 小游戏生产 Agent

规模化生产：两周上线 6 款小游戏

从 7 月开始，我们用两周时间，通过这套 Agent 完成并发布了 6 款小游戏。其中最快的一款在 Agent 平台内走完整个生产流程约 1 小时；算上测试、验收和上线流程，从创意到正式上线仅用 2 天。更重要的是，参与生产的人也不再局限于开发同学，测试、PD 和运营同学也可以用自己的 idea 制作小游戏。**不仅提升了生产效率，也在改变原有的生产关系，让更多角色从需求提出者、协作者转变为直接的生产者。这初步验证了小游戏生产 Agent 在不同角色的可用性，也证明了它规模化生产的潜力。**

小游戏	平台
忍者跳跃	

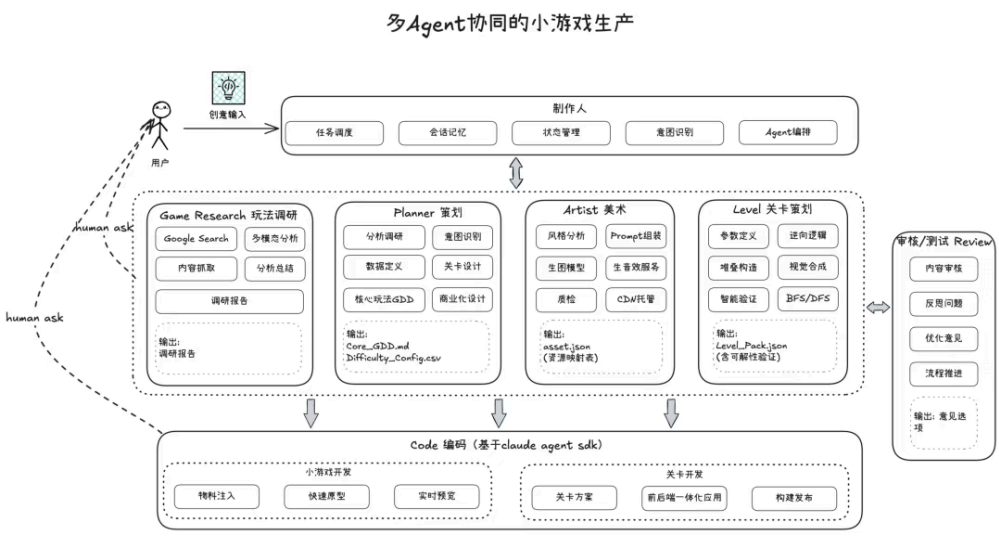
盖楼小游戏	
飞机大战	
贪吃蛇大作战	
打地鼠	

祖玛	

系统架构：Producer 统一调度 Multi-Agent

生产系统由两大技术栈协同驱动：

- **LangGraph**：负责全局流程编排与多 Agent 协调。
- **Claude Agent SDK（tarot-code-agent）**：负责在远程容器中执行真实的代码生成。



以 Producer（制作人） 为核心调度节点，协调 6 个专业 Agent。当用户输入一个创意后，Producer 会规划出一条完整的生产链路，按阶段依次推进：Gameplay Research → Planner → Art Designer → Game Coder → Level Designer → Level Coder。

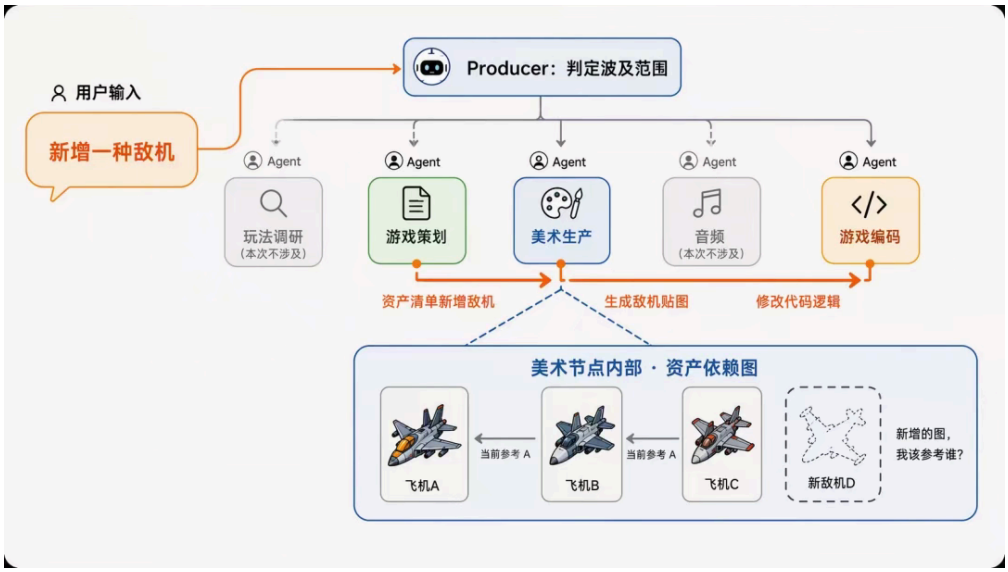
设计决策	说明
Producer 统一规划	所有路由决策由 Producer 通过 Structured Output 统一做出，避免把判断散落在各条边逻辑里
设计 → 审核 → 确认	设计类 Agent 产出后交付给人审核
上下文自动串联	策划方案、资产清单、关卡数据结构等进入全局状态，下游 Agent 可以直接使用
Human-in-the-loop	选图、方案确认、资产审核等关键节点用户可参与决策

多 Agent 架构详解

- Producer（制作人）— 全局调度中枢

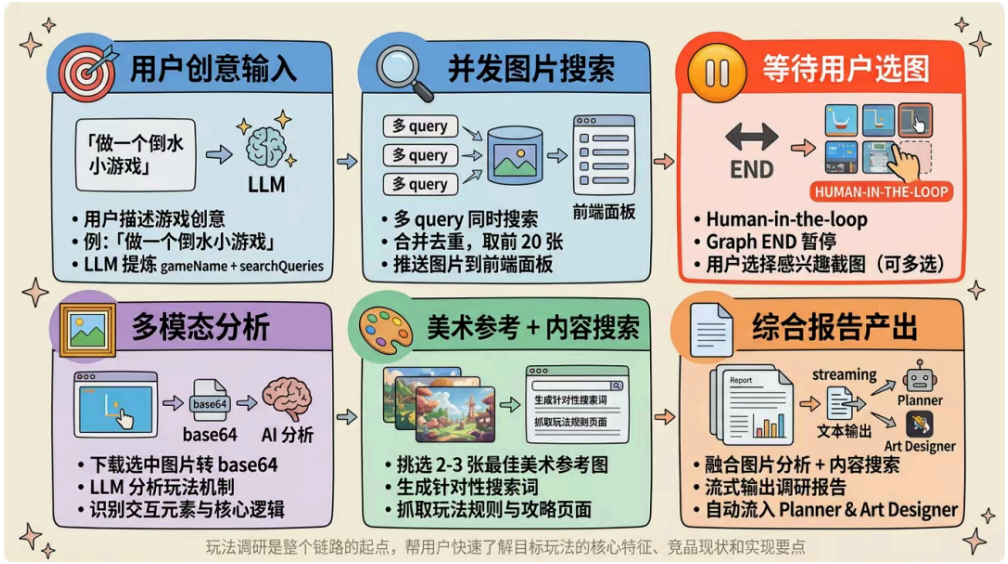
Producer 不直接生产任何内容，只专注于理解需求、规划链路、调度 Agent。当用户输入一个创意后，Producer 会分析当前项目状态，规划出一条完整的生产链路并按阶段推进。当用户在中途介入修改时，Producer 会动态的调整后续计划。

比如用户说 "新增一种敌机"，Producer 会先调度策划 Agent 更新资产清单，再调度资产 Agent 生成美术资产，最后调度开发 Agent 执行代码开发。



- Gameplay Research（玩法调研）— 从创意到调研报告

玩法调研是整个链路的起点。一个创意往往是模糊的，用户提出 "做一个倒水小游戏" 可能对应着写实物理模拟、卡通益智闯关、解压 ASMR 等截然不同的游戏类型。调研 Agent 的核心任务是**帮用户快速从发散收敛到明确的玩法方向。**

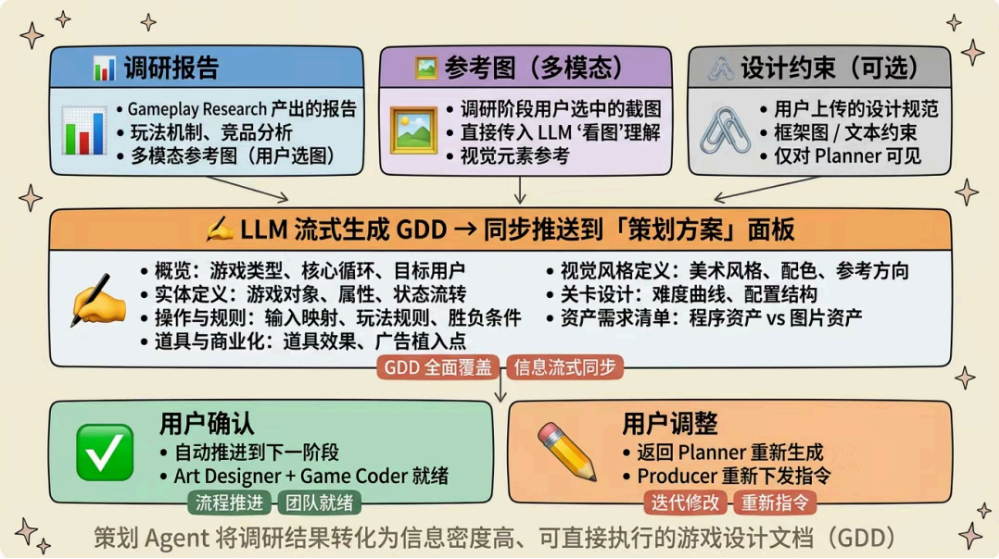


- **从发散到收敛**：不要求用户用文字精确描述想要的玩法，而是让用户从真实的游戏截图中选择。
- **游戏规则提炼**：直接让 LLM 分析用户选中的游戏截图，从视觉中提取出核心玩法机制。这些提炼的规则会被用作内容搜索的关键词，精准定位到详细的玩法规则和通关攻略。
- **筛选美术资产模版**：调研阶段会从选中图片中筛选出最适合作为美术参考的 2–3 张，贯穿到后续 Art Designer 的生成流程。

小游戏	玩法调研
飞机大战	
忍者跳跃	
麻将消消乐	

- Planner（策划 Agent）— 策划方案生成

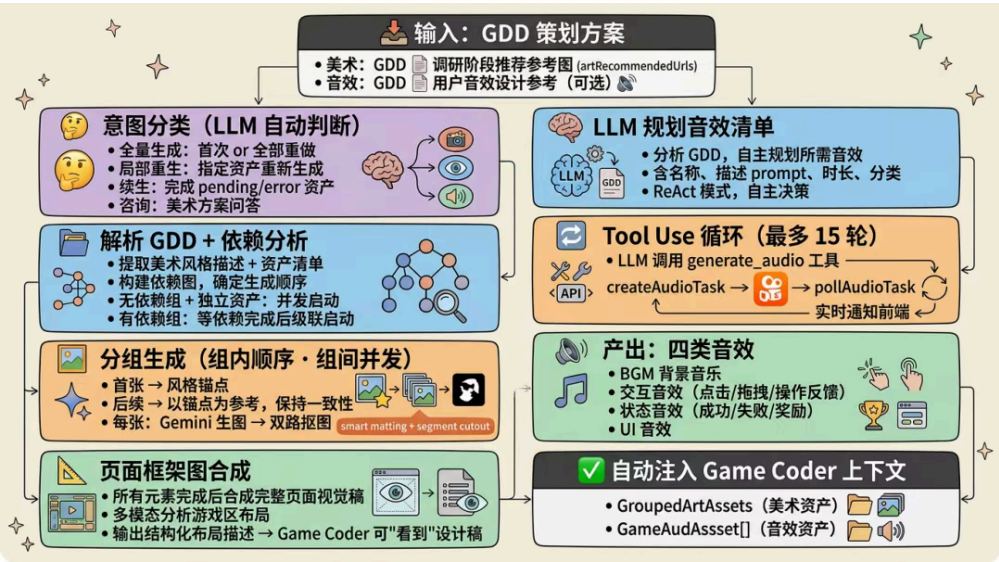
策划 Agent 负责将调研结果转化为一份**信息密度高、可直接使用**的游戏设计文档（GDD）。



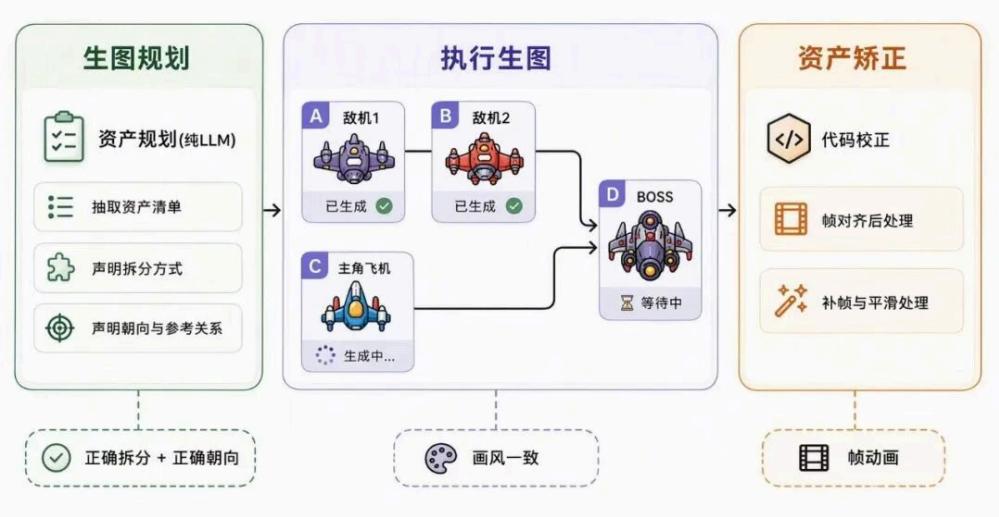
- **信息密度优先**：GDD 中每一段描述都必须能被下游 Agent 直接消费，游戏实体和操作规则给 Game Coder 实现玩法、资产清单给 Art Designer 生图、关卡设计给 Level Designer 定配置结构。
- **两类资产需求**：程序资产包括游戏实体、动画状态机、粒子特效等需要代码实现的对象；素材资产包括角色立绘、背景、UI 图标、音频等需要生成的素材。
- **定义美术风格**：策划阶段会先确定整体美术风格、主色调和参考方向，比如“扁平卡通”、“像素复古”或“3D 写实”，作为 Art Designer 生图时的全局约束。这避免了美术资产风格不统一、后期返工的问题。

- Art & Audio Designer（资产 Agent）— AI 生图、生音频

美术和音效 Agent 共同负责将 GDD 中的素材资产需求转化为实际可用的图片和音频文件。两者都属于执行类 Agent，直接输出文件，不再多写一层方案。

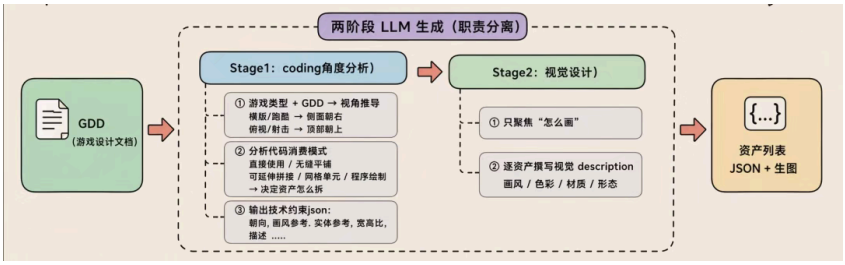


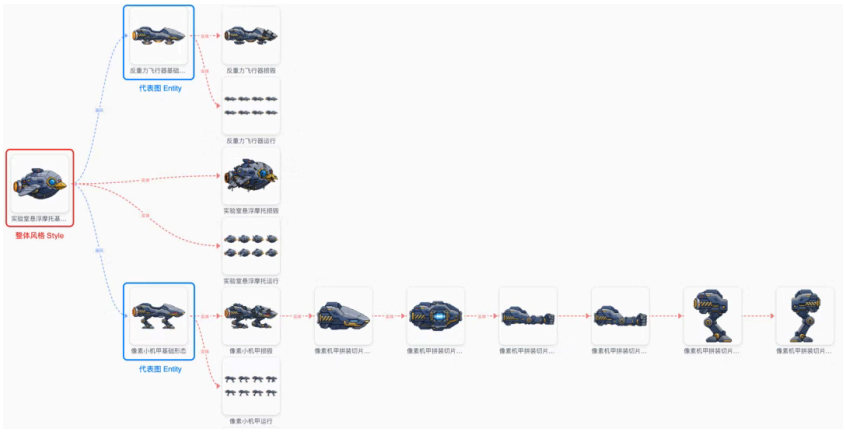
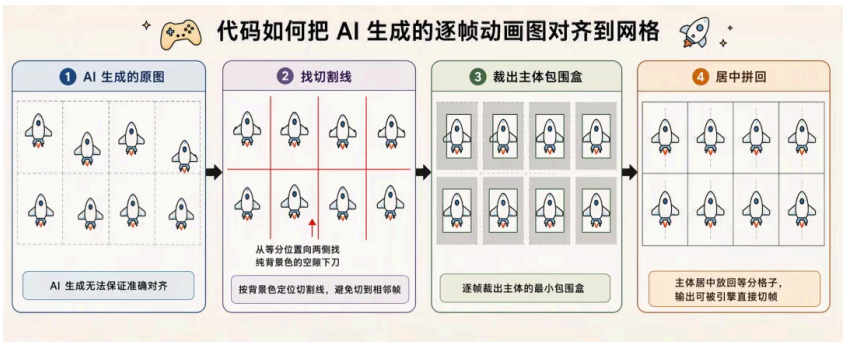
美术资产设计



美术资产最难的不是“能不能生成图片”，而是生成结果能否稳定进入工程。我们用了几层约束保证美术资产可用：

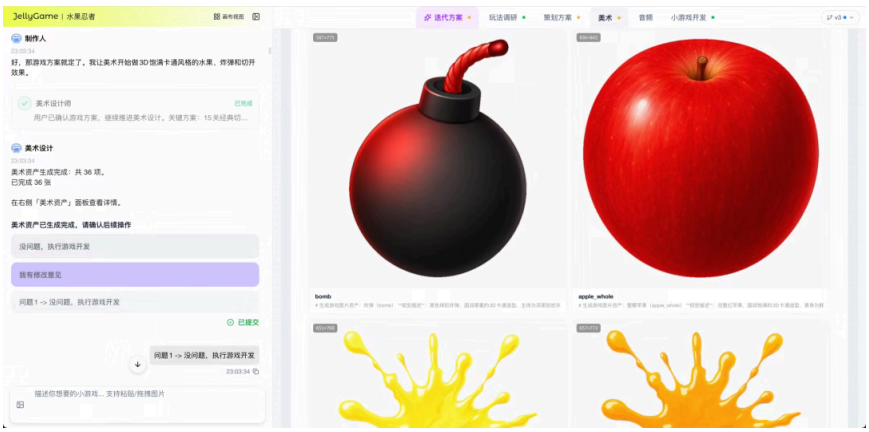
- **保障风格一致性**：
  a. 基准风格 Base：使用调研阶段推荐的风格参考图；
  b. 整体风格 Style：使用首张生成的素材作为锚点，后续资产以此为参考图；
  c. 代表图 Entity：有多种状态/关系的游戏对象，使用标准态作为参考图。
- **页面框架图 & 布局分析**：所有元素完成后，系统合成页面框架图，再用多模态模型分析游戏区布局，输出结构化描述。Game Coder 能读到页面的布局意图，而不是只拿一组零散图片和文字猜位置。
- **四种执行模式**：首次全量生成、用户指定局部重生、续生失败资产、回答美术咨询。模式由 LLM 做意图分类，局部修改不必每次从头生成。

流程	说明
资产规划	 资产规划属于决策环节，由 LLM 根据游戏设计文档列出完整资产，包括朝向图、拆分图等，实现先规划再生图的模式。

美术资产生成	 通过两类锚点，保证素材风格一致性。
美术资产矫正	 用工程能力处理 AI 生图无法精准对齐的问题。

音频资产设计

- **LLM 自主规划 + 工具调用**：音效 Agent 采用 ReAct 模式，LLM 分析 GDD 后自主决定生成哪些音效，灵活应对不同游戏类型的音效需求。
- **按类别组织**：音效按 BGM、交互音效、状态音效、UI 音效分类输出，Game Coder 可直接按场景引用。

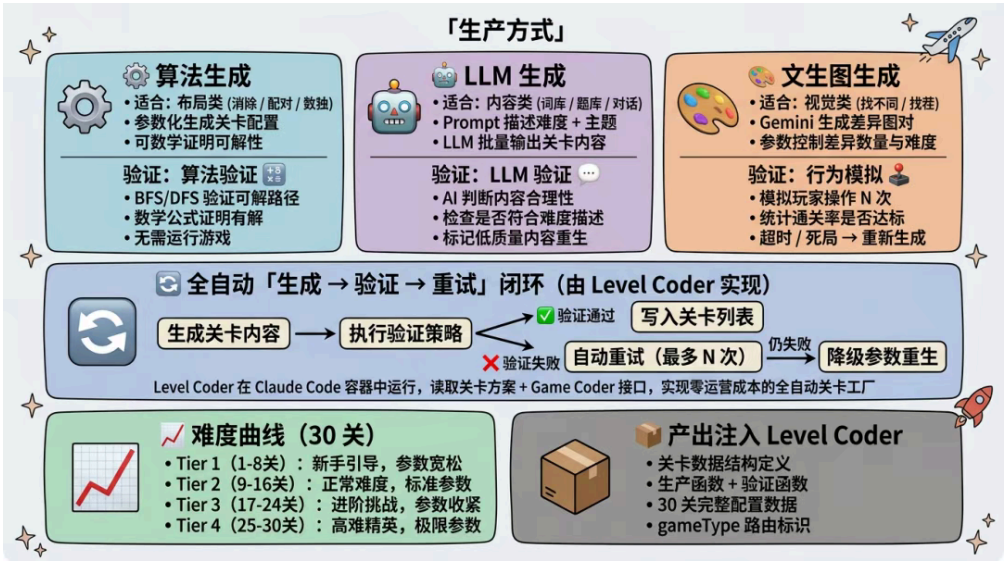
游戏	美术资产
水果忍者	

祖玛	
流沙方块	

- Level Designer（关卡设计 Agent）— 设计关卡工厂而非关卡

关卡策划 Agent 不逐一设计每一关，而是定义数据结构、生成算法、验证策略和难度曲线。

方案的核心是让每一关都为确定性产物：由 算法 + 参数 + Seed 共同生成，AI 看得懂、改得动、改完可验证。



关卡生成模式

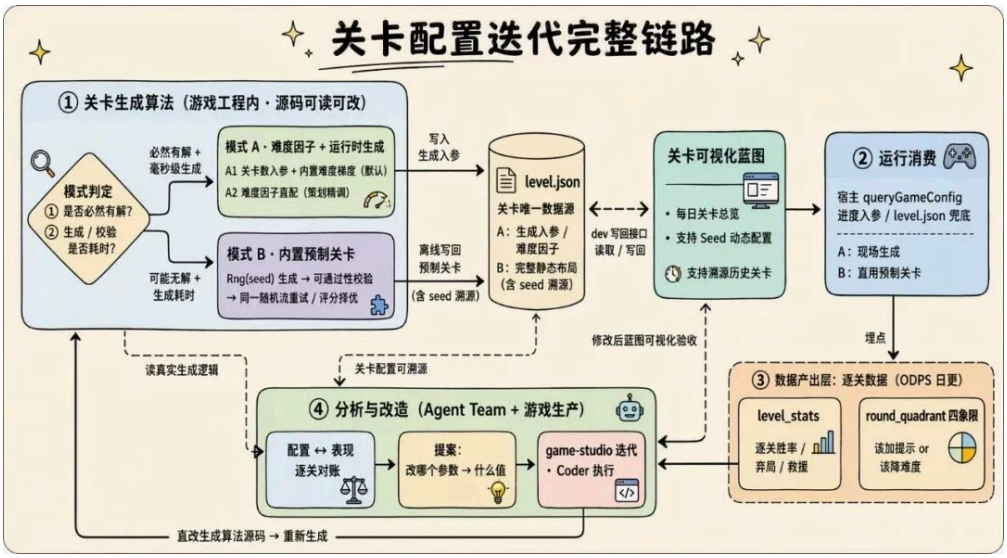
模式的选择根据两个判断：

- **关卡生成后是否必然有解？**
- **生成 & 可解性判定是否耗时？**

模式	难度定义	适用游戏类型
A · 难度因子配置	• 代码内置难度梯度，按进度派生所有难度因子； • 运行时根据 `Rng(seed)` 现场生成关卡内容；	• 飞机大战 等轮次闯关类； • 水果忍者 / 贪吃蛇 等操作型闯关类；
B · 内置预制关卡	• 关卡生成器离线生成：`Rng(seed)` 生成候选关卡，经过可通过性校验，不合格则重试，再按评分择优； • 运行时只消费静态数据，生成链路由构建配置隔离，不进入线上 MF 包；	• 忍者跳跃 / 疯狂喷气机 等地图跑酷类； • 数织 / 倒水 等推理求解类；

两种模式共同遵守一条 MF 契约：宿主默认只下发当前关卡数 & 总关卡数，游戏依据通关进度在本地动态生成关卡 或 直接使用内置关卡。

关卡迭代如何自闭环

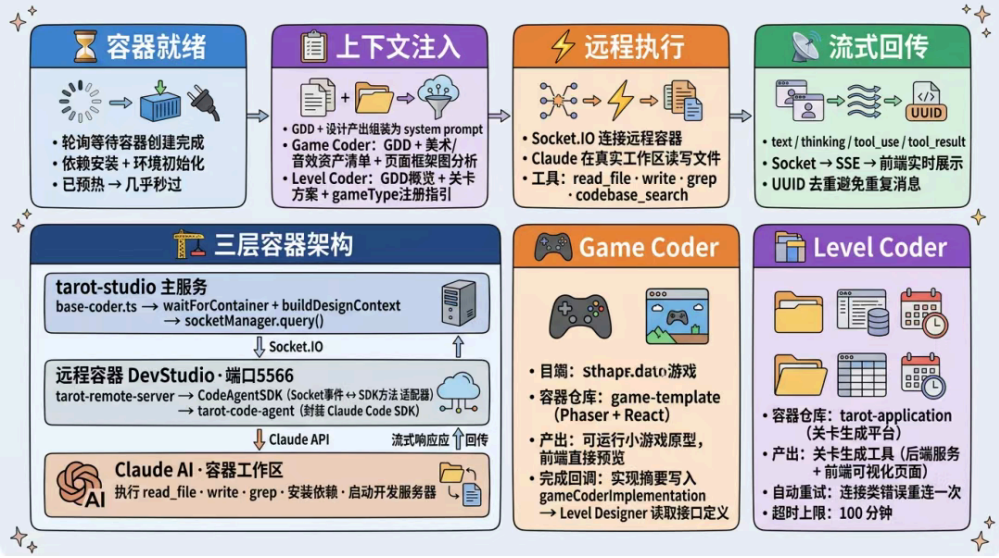


确定性生成让关卡具备三项后续迭代所需的能力：

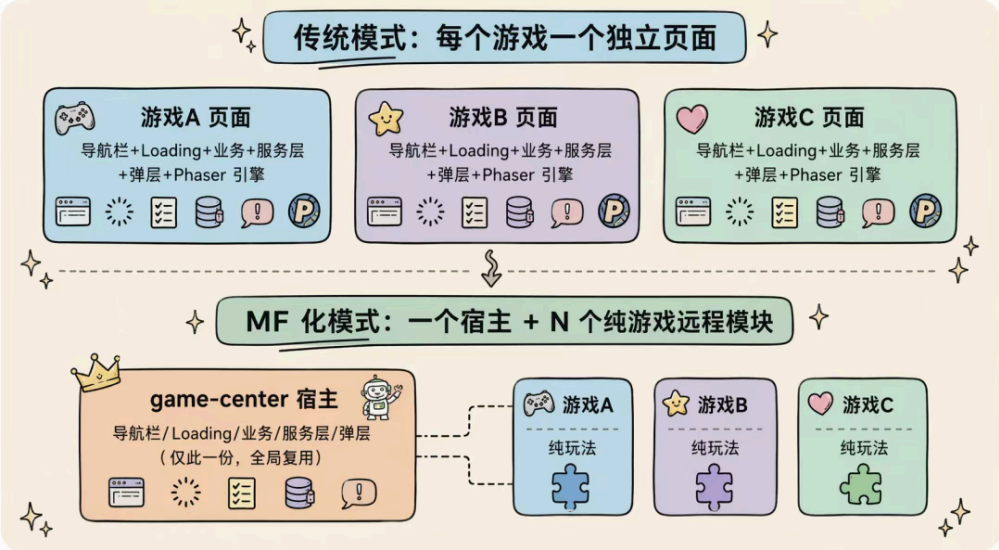
- **可溯源**：关卡是 算法 + 参数 + Seed 生成的确定性产物；任意一天、任意一关都能还原生成过程，问题关卡可以通过 Seed 和难度因子精确复现。
- **可改造**：
  - LLM 分析层拿到真实生成代码，而不只是配置值，因此提案可以明确到参数及目标值；
  - LLM 执行层修改源码后，可通过蓝图可视化和 Seed 逐关 diff 回归。
- **可验收**：蓝图与运行时复用同一条生成链路，蓝图中看到的关卡就是玩家实际进入的关卡。

- Game Coder & Level Coder — 远程容器编码

两个 Coder 是整个链路中唯一需要真实开发环境的 Agent。它们不只输出代码文本，还要读写文件、安装依赖和启动开发服务器，这由三层容器架构支撑：

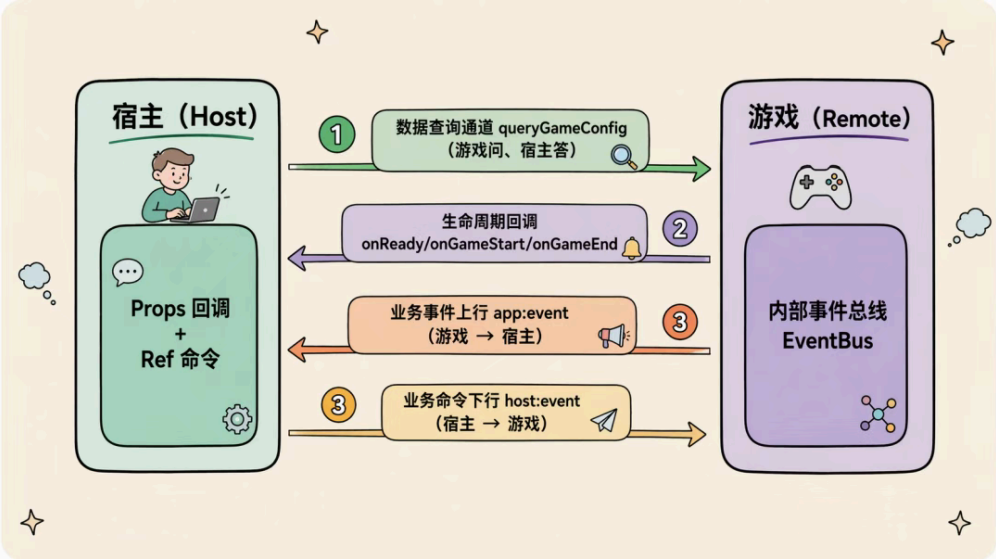


MF 场景化：模块联邦，动态可插拔



场景	传统模式的缺点（一个 H5 页面一个游戏）	MF 模块联邦的优势
业务逻辑 & 应用外壳	• 每个游戏工程重复实现同一套外壳	• 收敛到宿主一份，全局复用
新增一个游戏	• 新建工程 + 造应用外壳 + 页面发布 • 工程同时包含业务逻辑与游戏逻辑，其中业务逻辑重复度 95%	• asset 应用类型：MF 构建 + remoteEntry 入口 • 游戏模板只包含游戏逻辑，只开发游戏玩法 • 只需新增 remoteEntry 入口配置，宿主零代码改动
更新某个游戏	• 该游戏页面整体重新构建、发版、审核	• 游戏独立构建部署到 CDN，宿主无感
更新业务逻辑	• 每个游戏页都需要进行开发、测试 & 发布	• 仅更新宿主的业务代码
游戏打开体验	• 页面级游戏，切换游戏需要页面。	• 场景内直接打开，无明显加载体感

- 宿主与游戏的通信协议



① 统一数据查询通道（`queryGameConfig`）

游戏运行时需要从宿主获取业务数据（如第 N 关的关卡配置）。

- 协议没有为每种数据设计一个接口，而是用**一个通用异步函数**承载所有数据请求；
- 宿主侧自由决定数据来源，游戏侧完全不关心数据从哪来，只负责消费；

② 标准生命周期回调（游戏 → 宿主，单向通知）

生命周期事件由模板统一封装回调函数，游戏在关键节点单向通知宿主：

部分生命周期事件	说明
`onReady`	游戏引擎就绪
`onGameStart`	一局游戏开始
`onGameEnd`	一局游戏结束，包含 `success` 通关字段和业务数据
`onError`	异常报错或用户关闭返回

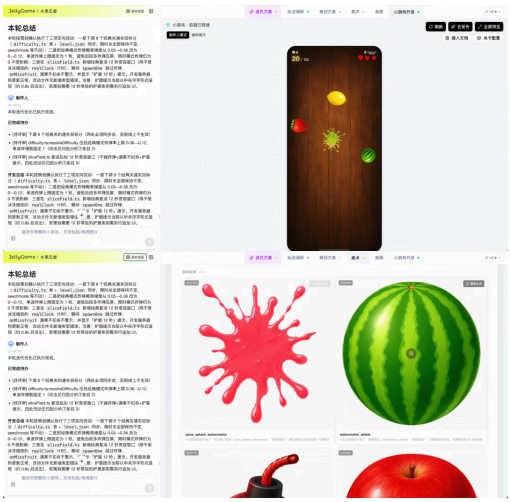
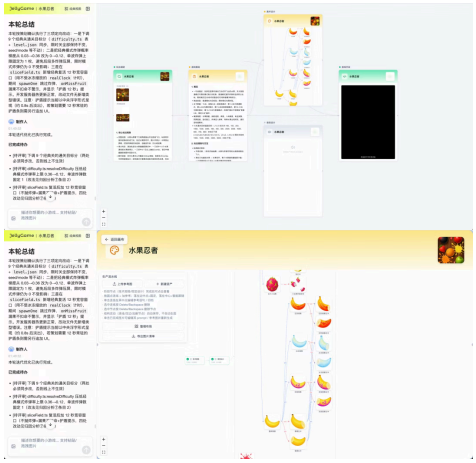
③ 双向事件协议

除了生命周期回调，游戏和宿主还需要传递大量业务自定义事件。双向事件协议提供上行和下行两条固定通道，所有额外的业务事件都走这两条通道，保障了协议的规范性和可扩展性：

- `app:event`（游戏 → 宿主）：游戏抛出的自定义业务事件。
  - 游戏侧 `emit('app:event', name, payload)` 发出，宿主侧通过 `onEvent(name, payload)` 回调接收。
- `host:event`（宿主 → 游戏）：宿主下发控制命令。
  - 宿主侧 `ref.sendHostEvent(name, payload)` 发出，游戏侧监听 `on('host:event', handler)` 接受。

画布视图 & 经典视图：两种交互形态

多 Agent 生产会话里同时存在调研报告、策划方案、美术资产、音频资产、关卡方案和游戏容器。单一交互方式很难兼顾当前环节的纵深操作与全局流程审查，所以平台提供经典视图和画布视图，并允许随时切换。

	经典视图	画布视图
页面视觉		
交互设计	对话 + 产物面板（Copilot 范式）	节点 + 连线（Workflow 范式）
信息组织	线性编排，聚焦单一环节	空间并列，展示全局拓扑
使用场景	首次生产、单环节审阅	全局审查、迭代修改、精细调整

关键优势与核心特点

- 架构层面：Workflow 与自主推理的融合

纯 Workflow 每一步固定、灵活性差，**纯自主 Agent** 全靠模型决策、不可控且 token 消耗大。我们通过 LangGraph + Claude Agent SDK 的配合，在两者之间找到了平衡点：

- **LangGraph 负责 "可控的流程骨架"**：生产链路的主干是确定的（调研→策划→美术→编码→关卡→关卡编码），每个阶段之间的流转、审核、暂停恢复都由结构硬性保证，不依赖模型的判断。这确保了即使模型偶尔"跑偏"，整体流程也不会失控。
- **子 Agent 内部充分发挥模型的自主推理**：在每个节点内部，模型拥有充分的自由度。Planner 可以自主决定 GDD 的章节深度和侧重点，Art Designer 可以自主分析依赖关系

和生成顺序，Level Designer 可以自主判断是否需要提问。流程不干涉节点内部的推理过程。

- **Human-in-the-loop 的时机精心设计**：不是每一步都问用户，也不是全程无人。只在三类关键节点暂停，其余环节全自动推进，既保证用户掌控权，又避免过度打断：
  - **方向选择**：调研阶段选图，决定玩法方向
  - **方案确认**：策划/关卡/美术产出后 Review
  - **质量验收**：代码完成后预览
- **两个引擎各取所长**：流程中的设计类 Agent 直接调用 LLM API，响应快、成本低；需要真实开发环境的 Coder 类 Agent 通过 Claude Agent SDK 在隔离容器中执行，拥有完整的文件系统和工具链。设计产物通过全局状态自动流入 Coder 的系统提示词，无需额外的 "交接" 环节。

- 子 Agent 层面：每个环节的针对性设计

- **玩法调研——以图代言，从发散到收敛**：用户说 "做一个消除类游戏"，可能指糖果传奇、俄罗斯方块、麻将连连看等截然不同的玩法。让用户用文字精确描述成本极高。我们反其道而行，先搜索、后选图，用户只需 "指" 出感兴趣的截图，选中的图片再经多模态分析提取玩法规则，精准定位到详细攻略，同时筛选出最适合的美术参考图贯穿到后续生图流程。
- **策划方案——信息密度最大化**：GDD 不在多，在于每一句话都能被下游 Agent 直接消费。实体定义给 Coder 写类、操作规则给 Coder 写逻辑、图片资产清单给 Art Designer 生图、程序资产清单给 Coder 实现、视觉风格定义约束全局美术方向。如果一段描述不能映射到具体的开发动作，就不写。
- **美术资产——生产线式的生图 Workflow**：不是简单地 "一个 prompt 生一张图"，而是一套完整的生图流程。GDD 解析提取需求清单 → 依赖分析确定生成顺序 → 分组并发 + 风格锚点保证一致性 → 每张图走 Gemini 生图 + 抠图 → 所有元素完成后合成页面框架图 → 多模态分析布局输出结构化描述给 Coder。音效流程上，LLM 分析 GDD 自主规划音效清单，通过 Tool Use 逐个调用可灵 AI 生成，按类别（BGM/交互/状态/UI）组织输出。
- **关卡设计——信息密度与可执行性并重**：关卡方案严格限定为 5 章（gameType、数据结构、生产方式、验证方式、难度曲线），每一章都是 Level Coder 的直接输入。Level Designer 优先基于 Game Coder 已实现的 TypeScript 接口来设计，确保方案产出即可落地。

- 工程基建：三层容器架构 + 固化框架与可变内容

我们通过构建三层容器架构，并在其中预置了包含 "固化层 + 可变层" 的壳应用，让 Coder Agent 只需关注游戏本身的差异化逻辑。这种分层带来两个好处：

1. **Coder 不用从零搭建工程基础设施**，只需往固化的框架里填充游戏特有的逻辑，大幅缩短编码时间；
2. **服务端能力完全复用**，通用的关卡查询、关卡推进、道具管理、广告打通等接口，新游戏接入时无需重复开发。

三层架构各司其职：

层级	职责
调度层	LangGraph 多 Agent 编排、设计产物管理、上下文组装、通过 Socket.IO 向容器发送编码指令
容器服务层	部署在每个容器内，CodeAgentSDK 将网络事件翻译为 Agent 方法调用，同时管理依赖安装、开发服务器、Git 操作
AI Agent 层	封装 Claude Code SDK，在真实工作区中执行文件读写、代码搜索、命令运行，像人类开发者一样工作

每个 Coder 节点对接一个独立的远程容器，容器内预装了对应的壳应用代码。壳应用的核心设计是**明确区分固化层与可变层**：

小游戏壳应用：

- **固化层**：Phaser + React 工程脚手架、Controller 分层架构（业务/游戏/音频）、通用 UI 框架（Loading/导航栏/功能面板）、状态管理机制、通用接口设计（关卡查询推进、道具 CRUD）、资源加载链路和事件总线
- **可变层**：游戏对象和管理器、主场景逻辑、具体 UI 内容和交互、关卡配置数据、道具效果、资源定义

关卡生成壳应用：

- **固化层**：Period → Day → Level 三级周期配置、通用服务接口（关卡/道具/广告链路）、前端路由框架和 gameType 注册机制、Schema 驱动的配置体系、构建发布流程
- **可变层**：具体游戏的关卡生成算法、验证逻辑、可视化页面、关卡 JSON 结构、参数曲线

05 小游戏运营：从数据到决策，再到自进化

规模化验证：12 款游戏、20+ 迭代

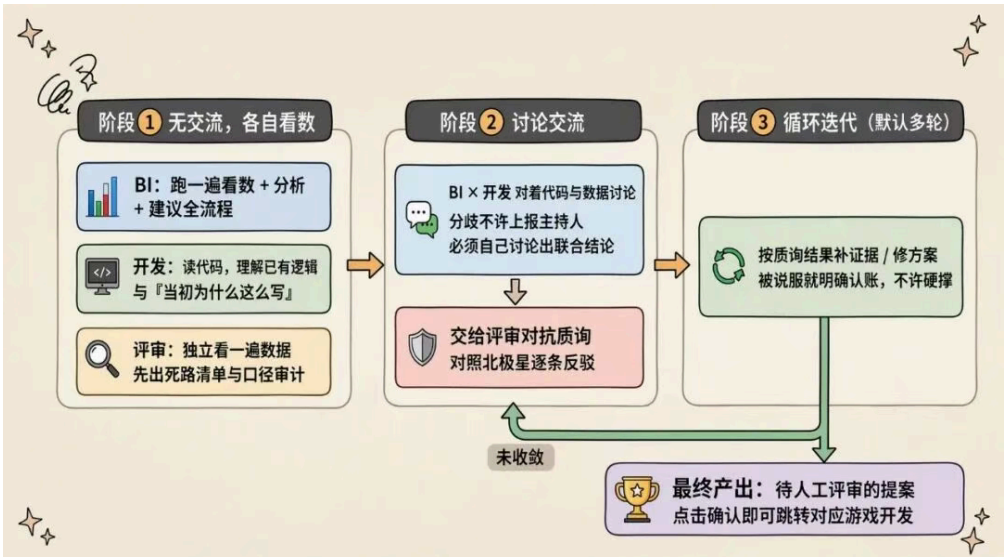
8 月底 Agent 自运营能力上线后，我们共上线 20+ 次游戏迭代和 1 款新游戏。首周先对线上 12 款小游戏做全面迭代，其中 95%+ 的迭代呈正向效果，40%+ 的迭代为全指标正向。因为分析和开发侧都已打通自动化流程，这 12 款游戏迭代从方案产出到开发上线只用了 3 天。

小游戏	迭代方案	迭代效果
趣味数独 **所有指标均正向**		
忍者跳跃 **所有指标均正向**		
一箭又一箭 **所有指标均正向**		

Agent Team：主持人 + 专家团的对抗模式

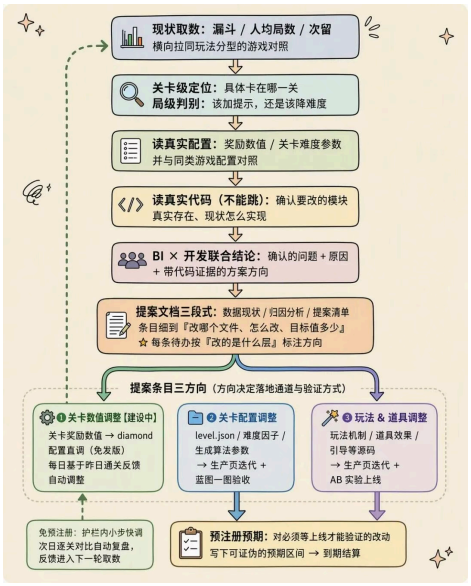
运营分析没有标准答案。单 Agent 容易把方案做得过于简单，把原因归到无法行动的客观因素上，或给出缺少证据、无法落地的结论。因此，我们选择采用“主持人 + 专家团”的多 Agent 模式。每个专家先独立看数，形成自己的上下文和证据链，避免过早锚定。不同任务有各自的系统 Prompt，明确步骤、门禁和已知死路，并在开工时强制注入。专家之间围绕具体方案互相论证，分歧在对话中解决，最后再由主持人收敛，不把未经处理的争论直接抛给用户。

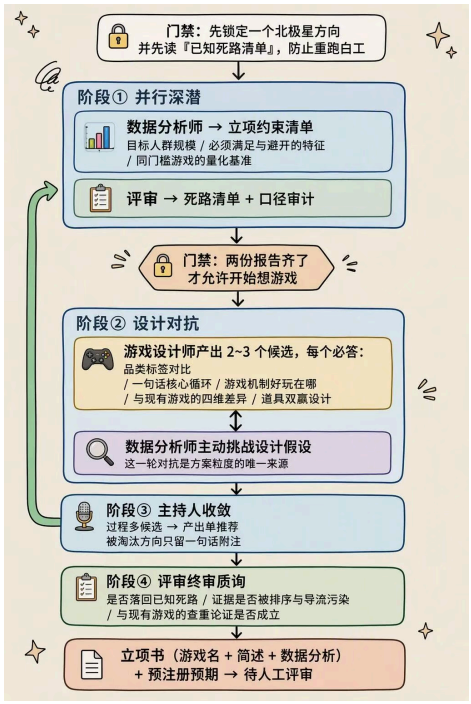
角色	职责	关注目标
主持人	• 启动 & 管理专家团 • 收敛结论 • 与用户对话	• 推进分析流程
数据分析师	• 分析数据 + 迭代日志 + 业务背景 • 定位异常并归因，提可执行提案	• 迭代目标：UV 提升 / 次留提升
开发工程师	• 读策划方案、真实代码 & 关卡配置 • 决策可行性 & 落地提案：接受 / 修正 / 驳回	• 可落地 • 风险可控 • 开发成本合理
游戏设计师	• 结合 BI 的数据分析结论提出 1–3 个游戏候选 • 和 BI 结对产出终选游戏的提案书	• 玩法差异化 • 双赢设计
评审	• 独立看数 • 质询与反驳：以北极星指标作为唯一准则	• 有效性 • 准确性 • 目标对齐



主要产出两类结果：

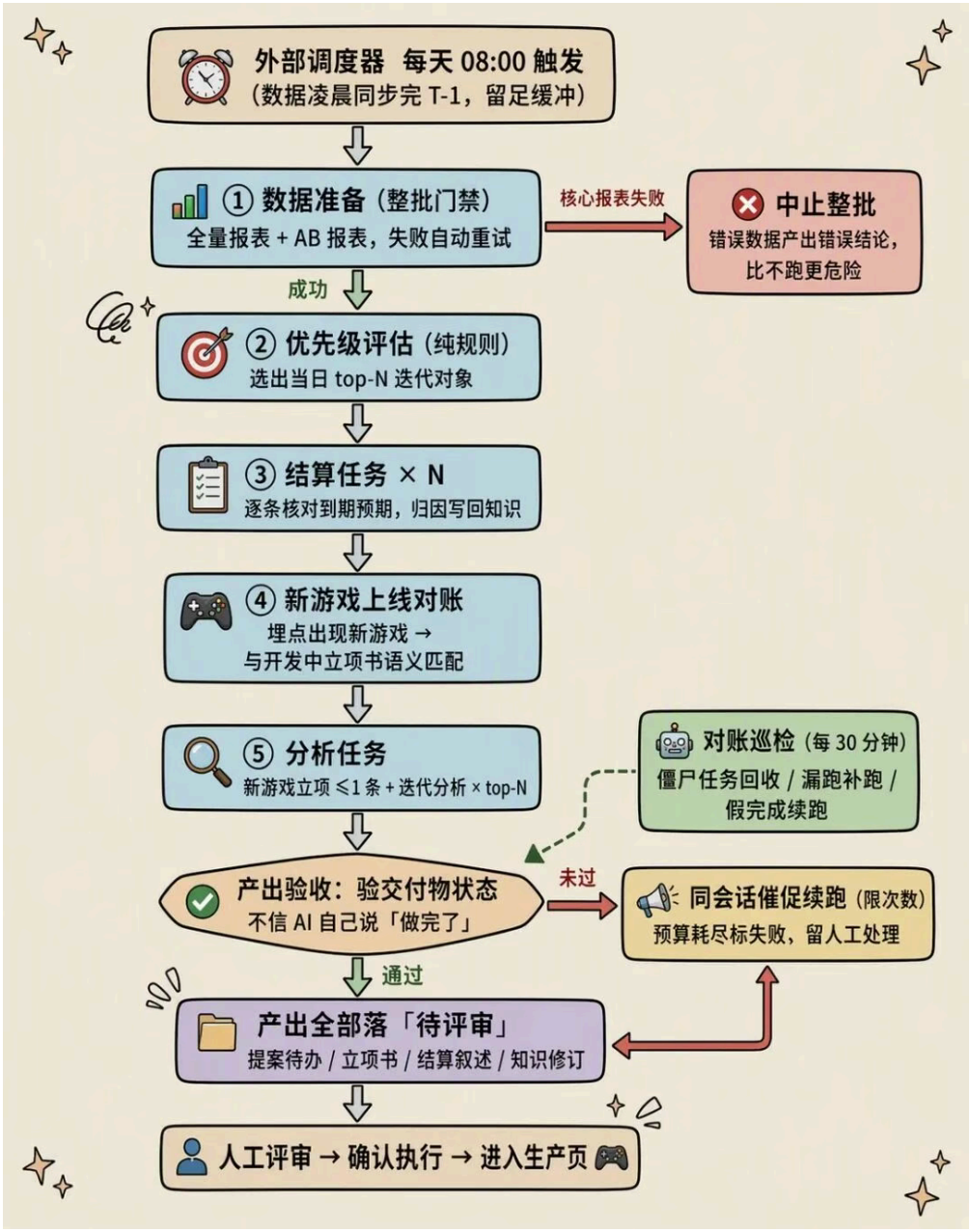
现有游戏迭代提案：	新游戏立项：





每日定时任务：让分析系统自己跑起来

对话式 Agent "问什么答什么"，但运营的日常工作方式不应该和 AI 有过多实时对话，而是以一个决策者的身份判断结论：前一天数据有什么变化，哪些实验预期已经到期，今天最值得改哪款游戏。流程相对固定后，我们把它编排成自动任务。



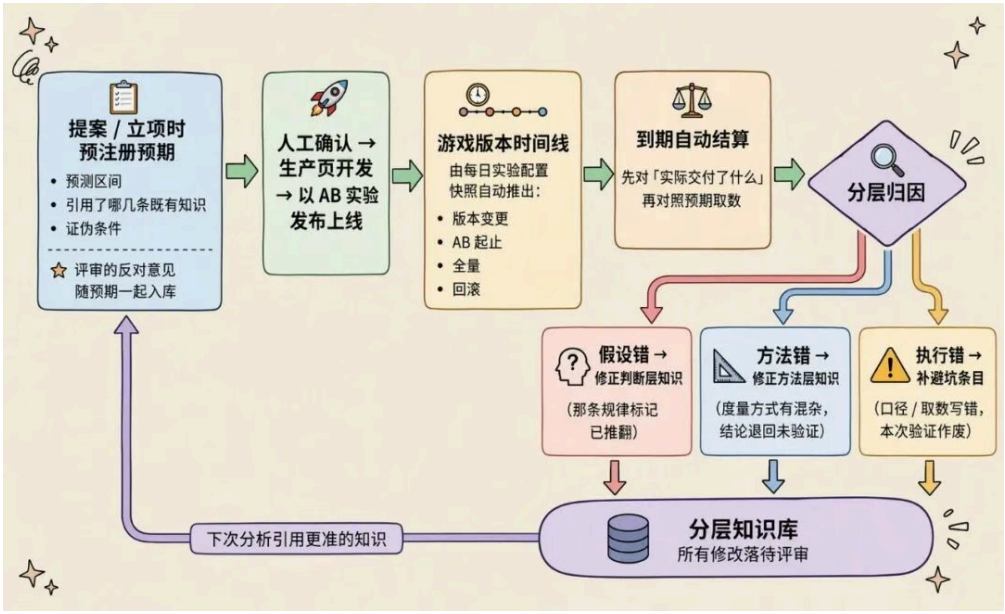
无人值守最需要避免的不只是明确的失败，还有把错误的结果输入待评审队列和知识库。为此，我们加了几道门禁：

- **数据整批验收：**核心报表构建失败，当天所有任务中止；宁可不跑，也不让错误数据生成结论。
- **验收真实状态：**会话正常退出不代表任务完成，系统检查立项书是否新建、提案是否更新、待结算清单是否清空。未通过时在同一会话中催促续跑，预算耗尽后才标记失败并交给人工。
- **自动巡检：**每半小时巡检一次，回收假死任务、补跑漏掉的批次，并纠正“看起来完成、实际没有交付物”的状态。
- **统一交付标准：**自动化只省掉中途等待同意的步骤。查重、读代码和证据要求与交互式流程一致，产物进入待评审队列，评审权仍然在人手里。

能力自进化：预期收益-真实数据回收

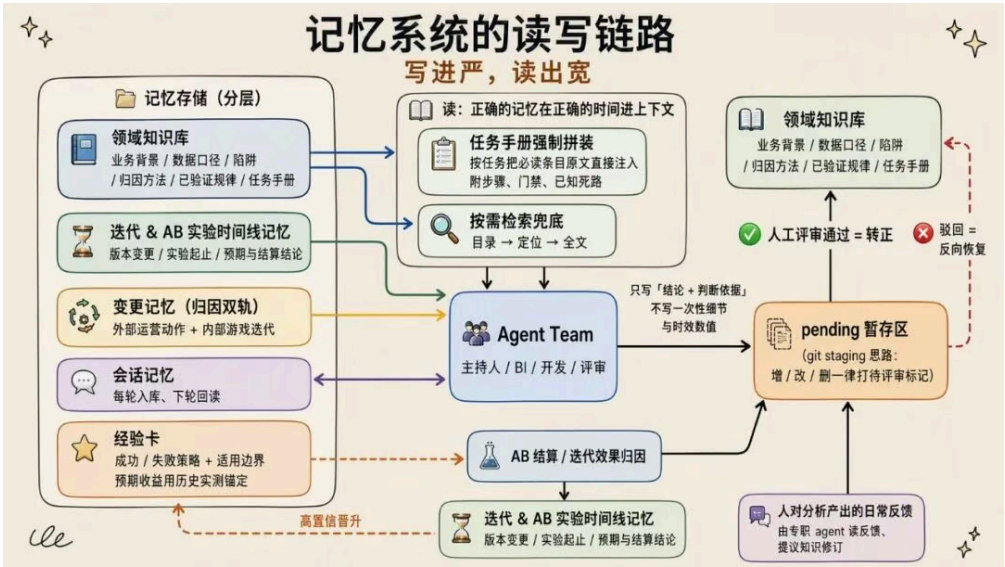
如果系统持续生成方案，它不会因为跑得更多就自动变好。我们把 Agent 的交付物重新定义为一次可证伪的“下注”：每个策略都要写明预测区间、引用了哪些知识，以及什么结果会证伪当前判断。

数据回收时，系统逐条对账预期与实际结果，并追溯本次判断引用的知识。如果知识本身有问题，就进入后续治理，而不是只把实验标记为成功或失败。



记忆系统：分层保存约束、经验、流程和事件

实验回收得到的经验需要合理保存，也要在合适的任务里取得出来。



我们采用四层记忆，把不同性质的信息放进不同成本、容量和检索方式的容器，让相关信息在需要时进入模型上下文。

分层	承载内容	本系统对应	如何消费
核心规则	• 稳定、高频、每次必须生效的规则 • 由人工审核的确定性内容	• 北极星指标 • 行动边界 • 诚实纪律 • 数据口径红线	• 常驻注入
知识	• 已沉淀的经验与教训 • LLM 可检索、自演化	• 已验证规律：按现象、机制、未验证推断分级 • 成败经验：含条件、限制、证据和反证 • 程序性方法	• 按游戏类型 & 任务需求动态匹配。 • 引用时携带匹配分数。
历史记忆	• 原始事件与证据 • LLM 提炼的历史摘要	• 分析轮次 • 运营动作的变更日志 • 内部改动的迭代记录 • 迭代与 AB 实验时间线	• 原始事件：充当唯一可直接引用的历史证据。 • 历史摘要：作为历史背景的补充。
会话记忆	• 会话中的所有内容	• 会话详情 • SDK 转录信息 • 本轮任务状态	• 中断重连：用于恢复会话的必要消息；

一条记忆的正文只保留“结论 + 判断依据”，不保存一次性细节和容易过期的时效数值。每条记忆还带有治理元数据：

- 作用域：适用于单款游戏，还是可以跨游戏复用。
- 来源证据：来自哪次结算、哪个会话。
- 证据等级：现象、机制或未验证推断；低等级内容禁止使用“显著”、“必然”等强断言。
- 置信度与状态：候选、生效或已失效。
- 版本链：可以回溯到生成这条记忆时的系统状态。

过去几个月，我们在淘天互动业务中跑通了小游戏从选型、生产到上线迭代的完整链路。但当前系统受游戏模式和关卡类型限制，优化主要以小游戏主体为主。

下一步，我们会把生产与迭代扩展到更广义的互动玩法。系统可以围绕业务目标和用户关系，组合抽奖、社交裂变、组队 PK、任务协作等机制，生成相应的规则、内容与体验，并根据用户反馈持续调整和验证。

评价这套能力，最终要看玩家是否愿意参与、是否觉得好玩、是否愿意再来，以及能否带来业务增量。我们希望逐步把一次性交付的互动项目，变成可以持续探索玩法、理解用户并优化效果的互动服务。

07 团队介绍

本文作者辰霁、成禹，来自淘天集团业务技术–营销&交易技术团队。团队承担淘宝天猫电商全链路交易与营销技术攻坚，致力于将 AI 技术能力与电商业务实践深度融合，通过技术创新持续驱动业务增长与用户体验升级。

在 AI 技术能力建设上，团队围绕互动玩法 & 体验，覆盖多 Agent 编排、上下文流转、全链路自迭代、数据决策闭环、记忆系统等多维链路，持续探索 AI 生产 & 运营互动玩法，尝试把玩法创意、内容生产、研发交付和运营迭代连接起来。在业务落地上，团队工作覆盖淘宝秒杀、淘宝特价版 App，主导了多个高价值项目，涵盖 618、双 11 等大促场景，将 AI 技术能力规模化渗透至秒杀、导购、交易、搜索、用增等核心业务场景。在协同创新上，小游戏 Agent 打破研发与业务角色之间的生产边界，让PD、运营、测试等角色从需求提出者转变为直接的生产者，在提升生产效率的同时重塑互动业务的生产关系，为规模化玩法创新注入持续动力。

拓展阅读

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法

大淘宝技术

大淘宝技术官方账号

964篇原创内容

公众号

阅读 6033

留言

写留言

还没有留言，来写首评